

OE-EC803B — Big Data Analysis

Semester VIII • 3L:0T:0P • 3 Credits

Module 0: Pre-requisite Computer Science Fundamentals

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Java Programming & OOP Foundations	2
1.1	Object-Oriented Programming (OOP) Principles	2
1.2	Java Collections Framework	2
1.3	JVM Memory Management	2
2	Structured Query Language (SQL) & Relational Algebra	3
2.1	Logical Execution Order of an SQL Query	3
2.2	SQL Joins	4
2.3	Subqueries and Common Table Expressions (CTEs)	4
3	Linux Command Line & Unix Text Processing	4
3.1	Core File Manipulation Commands	5
3.2	Redirections and Pipelines	5
3.3	Unix Text Processing Tools	5

1. Java Programming & OOP Foundations

Distributed frameworks in the Hadoop Ecosystem are heavily built on Java. A strong grasp of Java's runtime environment and Object-Oriented Programming (OOP) paradigms is essential to understand HDFS mechanics and MapReduce workflows.

Java Virtual Machine (JVM)

The **Java Virtual Machine (JVM)** is an abstract computing machine that provides a runtime environment in which Java bytecode can be executed. It acts as an engine that drives the execution of Java code, converting bytecode to machine-specific instructions and managing system memory.

1.1 Object-Oriented Programming (OOP) Principles

Java uses four primary OOP pillars which map directly to distributed programming:

- **Encapsulation:** Bundling fields and methods within a single class and restricting direct access (using access modifiers like 'private', 'protected', 'public') to protect the internal state of objects.
- **Inheritance:** Allowing a class (subclass) to inherit fields and methods from another class (superclass). In Hadoop, custom mappers and reducers inherit from base API classes.
- **Polymorphism:** The ability of an object to take on many forms. Used in Hadoop when custom serialized types implement interface contracts (e.g., overriding the 'write()' and 'readFields()' methods of the 'Writable' interface).
- **Abstraction:** Hiding implementation details and showing only functional interfaces using 'abstract' classes and 'interfaces'.

1.2 Java Collections Framework

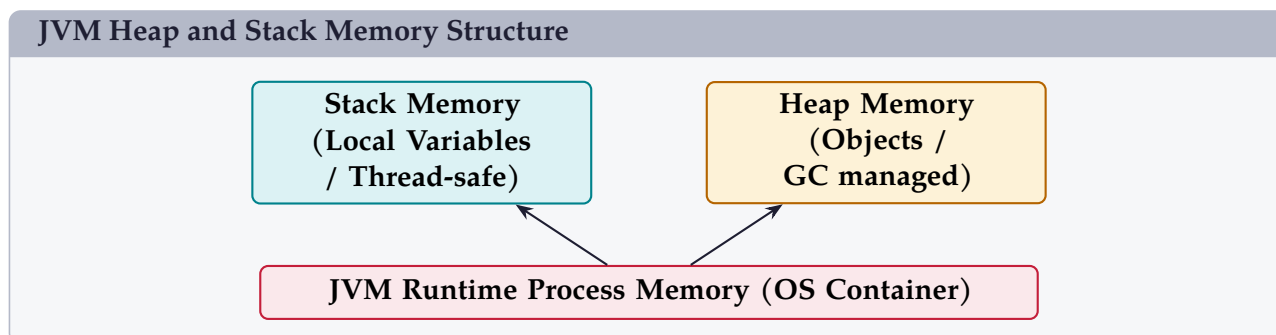
In MapReduce, intermediate values are held in collection data structures. The framework provides three primary categories of collections:

Collection	Key Characteristics	Underlying Structure	Hadoop Use Case
'ArrayList'	Ordered, indexable list. Allows duplicates. Resizes dynamically.	Resizable Array	Gathering list of values associated with a single key in Reducer.
'HashSet'	Unordered collection. No duplicates allowed.	Hash Table	Filtering out duplicate records or keys during processing.
'HashMap'	Key-value pairs storage. Unique keys. Fast search.	Hash Table + Linked List	Local in-memory caching and Map-Side Joins.

1.3 JVM Memory Management

The JVM splits its runtime memory into distinct regions. Understanding this is key to debugging Out-Of-Memory (OOM) errors in Hadoop:

- **Stack Memory:** Stores local variables and method execution frames. Allocation and deallocation are managed automatically.
- **Heap Memory:** Stores all objects instantiated via the 'new' keyword. Managed by the JVM Garbage Collector (GC). If MapReduce buffers grow too large, Heap memory exhaustion triggers a JVM crash.



2. Structured Query Language (SQL) & Relational Algebra

Data warehouse systems like Apache Hive use HiveQL, which is a dialect of SQL. Distributed querying translates SQL declarations into computational graphs.

Relational Databases vs. Hadoop Queries

Traditional databases use **Schema-on-Write**, validating data constraints during insertion. SQL engines on Hadoop (like Hive and Big SQL) use **Schema-on-Read**, overlaying structure on top of raw file streams (e.g., text, CSV, ORC) at query execution time.

2.1 Logical Execution Order of an SQL Query

While SQL queries are written starting with the 'SELECT' clause, the SQL compiler executes operations in a different logical sequence:

Logical SQL Execution Order

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

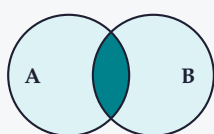
- **FROM/JOIN:** Identifies target datasets and links them.
- **WHERE:** Filters rows based on non-aggregated predicates.
- **GROUP BY:** Groups rows based on key values.
- **HAVING:** Filters groups using aggregate functions (e.g., 'HAVING count(*) > 5').
- **SELECT:** Projects fields and calculates functions.
- **ORDER BY:** Sorts output (very expensive in distributed systems as it forces global sorting).

2.2 SQL Joins

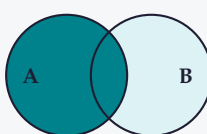
Joins are fundamental operations that merge datasets horizontally on a matching key column:

- **Inner Join:** Returns records that have matching values in both tables.
- **Left (Outer) Join:** Returns all records from the left table, and matched records from the right table. Fill with 'NULL' on the right side if no match.
- **Right (Outer) Join:** Returns all records from the right table, and matched records from the left table.
- **Full (Outer) Join:** Returns all records when there is a match in either left or right table.

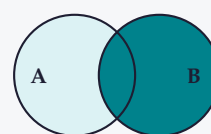
SQL Joins Visual Representation



Inner Join



Left Join



Right Join

2.3 Subqueries and Common Table Expressions (CTEs)

- **Subquery:** A query nested inside another SQL statement. It can return scalar values, single columns, or tables (e.g., 'SELECT * FROM emp WHERE salary > (SELECT AVG(salary) FROM emp)').
- **Common Table Expression (CTE):** A temporary named result set defined using the 'WITH' keyword, which improves readability compared to nested subqueries:

Common Table Expression (CTE) Syntax

```
WITH DeptAvgSalary AS (  
    SELECT dept_id, AVG(salary) AS avg_sal  
    FROM employees  
    GROUP BY dept_id  
)  
SELECT e.name, e.salary, d.avg_sal  
FROM employees e  
JOIN DeptAvgSalary d ON e.dept_id = d.dept_id;
```

3. Linux Command Line & Unix Text Processing

Hadoop CLI commands mimic standard Linux file system operations. Exposure to Unix text processing tools is essential for filtering, parsing, and verifying datasets before importing them.

Unix Streams

Unix systems model communications via three standard input/output channels (streams):

- **stdin (0):** Standard Input stream (default: keyboard).
- **stdout (1):** Standard Output stream (default: terminal screen).
- **stderr (2):** Standard Error stream (default: terminal screen).

3.1 Core File Manipulation Commands

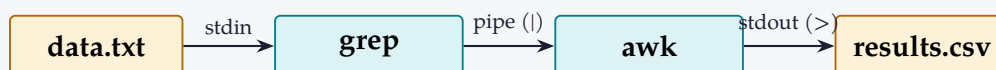
- **'cd <dir>':** Change current working directory.
- **'ls -la':** List directory contents showing file size, permissions, and hidden files.
- **'mkdir -p <path>':** Create directories recursively without throwing errors if they exist.
- **'cp -r <src> <dest>':** Copy files and directories recursively.
- **'mv <src> <dest>':** Move or rename files.
- **'rm -rf <path>':** Recursively force delete files or directories.
- **'chmod 755 <file>':** Modify file read-write-execute permissions (rwxr-xr-x).

3.2 Redirections and Pipelines

Commands can output data to files or other commands instead of printing to the terminal:

- **Redirections:** Redirect outputs or inputs to files.
 - **'>':** Redirects 'stdout' to a file (overwriting existing content).
 - **'>>':** Appends 'stdout' to a file.
 - **'2>':** Redirects 'stderr' to a file.
- **Pipes (|):** Passes the 'stdout' of one command as the 'stdin' of the next command.

Pipeline Data Flow



3.3 Unix Text Processing Tools

Hadoop Streaming allows using Unix programs as mappers and reducers. The core tools are:

- **'cat':** Concatenates and displays file contents.
- **'grep <pattern>':** Searches for regular expression pattern matches within files.
- **awk:** A pattern scanning and processing language. Heavily used to extract columns (e.g., `awk '{print $2}'` extracts the second space-separated field).
- **'sed':** A stream editor for parsing and transforming text (e.g., substitute strings).
- **'sort':** Sorts lines of text files alphabetically or numerically.
- **'uniq':** Discards duplicate adjacent lines. Needs data to be pre-sorted.

Common UNIX Text Processing Pipeline

To find the top 5 most frequent error types from a log file:

```
cat server.log | grep "ERROR" | awk '{print $5}' | sort | uniq -c | sort -nr | head -n 5
```

Explanation: Step 1: Print file → Step 2: Filter lines containing "ERROR" → Step 3: Extract the fifth column → Step 4: Sort columns → Step 5: Count occurrences of unique elements → Step 6: Sort counts numerically in descending order → Step 7: Print top 5 lines.

Quick Revision — Module 0

Java Collections, SQL Queries & Unix Pipelines | OE-EC803B

One-Line Definitions

- **Polymorphism:** The OOP capability to invoke object-specific methods via interface abstractions.
- **Schema-on-Read:** A data schema validation approach where structure is applied only when reading files.
- **Common Table Expression (CTE):** A temporary named result set defined locally within an SQL query block.
- **Unix Pipe:** A mechanism that directs the standard output of one command to the standard input of another.
- **Garbage Collector (GC):** A background JVM process that automatically reclaims memory by deleting unreferenced objects.

Frequently Asked / Viva Questions

Q1. Why is Java memory management critical in Hadoop MapReduce jobs?

Custom mapper and reducer code instantiates objects inside JVM heap containers. If collections (like lists or hash maps) grow indefinitely without boundary checks, JVM Heap runs out of memory, leading to job failure.

Q2. Compare SQL's WHERE and HAVING clauses.

The 'WHERE' clause filters individual rows before grouping operations occur. The 'HAVING' clause filters aggregated groups after the 'GROUP BY' clause has been processed.

Q3. Explain the utility of SQL Subqueries vs. CTEs.

Subqueries are nested expressions that can make queries complex and hard to read. Common Table Expressions (CTEs) define clear, modular virtual tables using 'WITH', facilitating compiler optimization and improving readability.

Q4. What does the command "chmod 755 script.sh" accomplish?

It sets file permissions: Owner receives full permissions (Read=4, Write=2, Execute=1 → 7). Group and Others receive Read and Execute permissions (Read=4, Execute=1 → 5).

Q5. Why must "uniq" be preceded by a "sort" command in Unix pipelines?

The 'uniq' command only matches and filters out adjacent duplicate lines. If duplicate records are scattered, 'uniq' will miss them. Pre-sorting groups duplicate lines together.

Q6. What is the difference between ">" and ">>" in Unix Shell?

Frequently Asked / Viva Questions (contd.)

'>' redirects standard output to a file, overwriting the file's current contents. '>>' redirects output to a file, appending the output at the end of the file.

Q7. Describe the JVM Stack vs. Heap memory allocations.

Stack allocation is temporary, storing thread execution states and primitive local variables. Heap stores dynamic class objects that require garbage collection.

Q8. What is the time complexity of looking up a key in a HashMap?

On average, 'HashMap' lookup has $O(1)$ constant time complexity. In the worst case (with hash collisions), it defaults to $O(n)$ or $O(\log n)$.

Q9. Explain standard error (stderr) redirection in shell.

Errors bypass standard output redirection. To capture error logs, redirect standard error using '2>' (e.g., 'command > out.txt 2> error.log').

Q10. What is the difference between Schema-on-Write and Schema-on-Read?

Schema-on-Write enforces schema definitions when writing data to relational tables, rejecting formatting violations. Schema-on-Read stores raw data files as-is, parsing formatting layouts dynamically during queries.

Comparison Snapshot

Foundational Paradigm Comparisons:

Feature	SQL Database Engine	Unix Command Line
Execution Paradigm	Declarative (describe <i>what</i> data to fetch)	Procedural (pipe <i>how</i> data flows through tools)
Primary Objective	Structured relational database querying	General-purpose file processing and system control
Data Structures	Tables, columns, index structures	Text streams, byte buffers, files
Scaling Pattern	Relies on query planners and indexes	Relies on stream processing buffers
Optimization	Handled by query optimizer engines	Handled manually by pipeline construction